Linux Troubleshooting & Tips

How to Use rpmbuild on Ubuntu to Create an RPM Package

sood

February 18, 2025

Ubuntu is designed around DEB packages and the APT package manager, but certain enterprise applications and vendor software require RPM packages. If you're working in a mixed Linux environment or need to build RPMs from source on Ubuntu, this guide will walk you through installing rpmbuild, setting up dependencies, and successfully creating an RPM package step by step.

Table of Contents

Why Build RPM Packages on Ubuntu?

Although Ubuntu primarily uses DEB packages, building RPMs can be useful for:

- Developers distributing software for Red Hat-based systems
- Testing software in mixed Linux environments
- Creating custom RPMs for enterprise applications

By using rpmbuild on Ubuntu, you can develop and test RPM packages without needing a separate RHEL-based system.

Installing rpmbuild on Ubuntu

Unlike Fedora or CentOS, Ubuntu does not include rpmbuild by default, so you must install it manually. To get started, update your package list and install the necessary dependencies to build an RPM package on Ubuntu without errors. This ensures you have all required tools, including compilers and build utilities.

Start by updating your package list:

```
sudo apt update
```

Next, install the rpm and build tools package:

```
sudo apt install rpm build-essential
```

The build-essential package includes gcc, make, and other necessary development tools. If you are working with C-based software, you may also need additional dependencies depending on your project.

To check if rpmbuild is installed, run:

```
rpmbuild --version
```

If the command returns a version number, the installation was successful.

Setting Up the RPM Build Environment on Ubuntu

Because Ubuntu is a Debian-based distribution, it lacks native support for RPM package building. To ensure a smooth process, you need to manually set up an RPM build environment on Ubuntu, creating essential directories and configuring system settings. This helps keep package files, source code, and dependencies properly structured for a successful build.

If you're new to scripting, understanding how to execute a Bash script on Ubuntu can help automate parts of the setup process.

1. Create a dedicated user for building RPMs to avoid permission issues:

```
sudo useradd -m -s /bin/bash makerpm
```

2. Switch to the new user account:

```
su - makerpm
```

3. Create the standard rpmbuild directory structure:

```
mkdir -p ~/rpmbuild/{BUILD,RPMS,SOURCES,SPECS,SRPMS}
```

4. Configure RPM macros by adding the following line to a new .rpmmacros file:

```
echo '%_topdir %(echo $HOME)/rpmbuild' > ~/.rpmmacros
```

This setup ensures that rpmbuild knows where to find the necessary

directories when creating the RPM package.

Downloading and Preparing the Source Code

Before you can build an RPM, you need source code. For this example, we'll use a simple "Hello World" program written in C.

Switch to the SOURCES directory:

```
cd ~/rpmbuild/SOURCES
```

Download or create a source archive. If you have a tar.gz archive of your source code, move it here. Otherwise, you can create one manually:

```
tar -czvf helloworld-1.0.tar.gz helloworld/
```

This archive will be referenced in the RPM spec file when defining how to build the package.

Creating the RPM Spec File on Ubuntu

An RPM spec file defines the package details, build process, and installation steps. It includes metadata like the package name, version, description, dependencies, and the commands to compile and install the software.

1. Navigate to the SPECS directory: `cd ~/rpmbuild/SPECS`
2. Create a new spec file using a text editor like nano: `nano helloworld.spec`
3. Add the following content to the spec file:

```
Name:          helloworld
Version:       1.0
Release:       1%{?dist}
Summary:       A simple Hello World program

License:       GPL
URL:           https://example.com
Source0:       helloworld-1.0.tar.gz

BuildRequires:  gcc

%description
A basic Hello World program written in C.

%prep
%setup -q

%build
gcc -o helloworld helloworld.c

%install
rm -rf $RPM_BUILD_ROOT
install -d $RPM_BUILD_ROOT/usr/local/bin
install -m 755 helloworld $RPM_BUILD_ROOT/usr/local/bin

%files
%defattr(-,root,root,-)
/usr/local/bin/helloworld

%changelog
* Mon Feb 18 2025 MakerPM - 1.0-1
- Initial RPM release
```

Breaking Down the Spec File

- **Name, Version, Release**: Defines the package metadata.
- **Summary & Description**: Provides an overview of the package.
- **Source0**: References the source archive.
- **BuildRequires**: Lists required dependencies.
- **%prep**: Extracts the source archive.
- **%build**: Compiles the program using gcc.
- **%install**: Moves the compiled binary to the correct directory.
- **%files**: Specifies the files included in the package.
- **%changelog**: Tracks package updates.

This spec file tells rpmbuild how to compile and package the software.

Building the RPM Package on Ubuntu

Once the spec file is ready, you can build the RPM package. If you prefer automating this step, you can use a script. Learn more about how to run a Bash script in Linux to streamline the build process.

1. Go to the SPECS directory:

```
cd ~/rpmbuild/SPECS
```

2. Run the rpmbuild command:

```
rpmbuild -ba helloworld.spec
```

This command compiles the source code, installs the files in a temporary

location, and packages everything into an RPM file.

3. Once completed, check the RPMS directory for the generated package:

```
ls ~/rpmbuild/RPMS/x86_64/
```

3.

If successful, you should see a file like **`helloworld-1.0-1.x86_64.rpm`**.

Installing and Testing the RPM Package on Ubuntu

Ubuntu does not use RPM packages natively, so you need additional tools to install and test the package.

Installing RPM Packages on Ubuntu

Since Ubuntu uses DEB packages, you need a tool like **`alien`** to convert and install the RPM file. First, install alien and other required utilities:

```
sudo apt install alien rpm
```

To convert the RPM package to a DEB format:

```
sudo alien -d ~/rpmbuild/RPMS/x86_64/helloworld-1.0-1.x86_64.rpm
```

This will generate a file like **`helloworld_1.0-1_amd64.deb`**. You can then install it using dpkg:

```
sudo dpkg -i helloworld_1.0-1_amd64.deb
```

Verifying the Installation

Check if the binary is installed:

```
ls /usr/local/bin/helloworld
```

If the file exists, run the program:

```
helloworld
```

It should print “Hello, World!” to the terminal.

Common Errors and Fixes When Using rpmbuild on Ubuntu

Even though Ubuntu can support RPM package building, you may encounter missing dependencies, path issues, or conversion errors. Below are the most frequent problems and how to fix them.

Error: “rpmbuild: command not found” on Ubuntu

If you get a **“rpmbuild: command not found”** error, it means rpmbuild is not installed or not in your system’s PATH.

How to Fix:

1. Verify the installation: `which rpmbuild` If no output appears, rpmbuild is missing.

2. Install it using: `sudo apt install rpm build-essential`
3. If installed but still not found, check your PATH: `echo $PATH` Ensure `/usr/bin/` is included. If not, add it manually: `export PATH=$PATH:/usr/bin/`

Error: rpmbuild missing dependencies on Ubuntu

RPM packages require build dependencies that Ubuntu does not install by default. If your build fails with a missing dependency error, follow these steps.

How to Fix:

1. Identify missing dependencies: `rpmbuild -ba package.spec 2>&1 | grep "error:"`
2. Install the required libraries: `sudo apt install package-name`
3. If a dependency is only available as an RPM, try converting it to DEB using Alien: `sudo alien -d package.rpm sudo dpkg -i package.deb`
4. As a last resort, manually extract the RPM file: `rpm2cpio package.rpm | cpio -idmv`

Error: Alien conversion fails when installing an RPM on Ubuntu

Alien sometimes fails when converting an RPM package to DEB due to scripting or architecture issues.

How to Fix:

1. Try forcing the conversion: `sudo fakeroot alien -d package.rpm`

2. If the package is architecture-specific (e.g., 32-bit on a 64-bit system), enable multi-arch support: `sudo dpkg --add-architecture i386 sudo apt update`
3. If Alien still fails, manually extract and install: `rpm2cpio package.rpm | cpio -idmv sudo cp -r * /usr/local/`

Error: "error: File not found" when building RPM

If rpmbuild cannot locate the source files, it may be due to an incorrect directory structure.

How to Fix:

1. Ensure your source archive is placed in the correct directory: `mv helloworld-1.0.tar.gz ~/rpmbuild/SOURCES/`
2. Check your spec file's Source0 path: `Source0: helloworld-1.0.tar.gz`
3. Verify the setup step: `%prep %setup -q` The `%setup` macro extracts files from the archive—without it, rpmbuild cannot proceed.

Error: Installed RPM package does not run on Ubuntu

After installing the RPM, you may get a **"command not found"** or a missing dependency error when trying to run the software.

How to Fix:

1. Check if the binary was installed: `ls /usr/local/bin/`
2. If missing, find where the package installed its files: `dpkg -L package-name`

3. If dependencies are missing, use `ldd` to check: `ldd /usr/local/bin/executable-name` Then install the missing libraries: `sudo apt install package-name`

FAQs

Why does Ubuntu use DEB instead of RPM?

Ubuntu is based on Debian, which uses the APT package manager and DEB format for easier dependency management. RPM is used by Red Hat-based distributions like RHEL, Fedora, and CentOS.

How do I install rpmbuild on Ubuntu?

Ubuntu does not include rpmbuild by default. Install it using **`sudo apt update`** followed by **`sudo apt install rpm build-essential`**. This will provide the necessary tools for building RPM packages on Ubuntu.

Can I build an RPM package on Ubuntu without errors?

Yes, but Ubuntu is not optimized for RPM packaging. You may face missing dependencies. Install Alien, RPM, and required libraries. Ensure you create the correct build directories and set up the rpmbuild environment properly.

How do I convert RPM to DEB on Ubuntu?

Use the Alien tool to convert RPM packages. Install it with **`sudo apt install alien`**, then run **`sudo alien -d package.rpm`**. This creates a DEB file that you can install with dpkg.

How do I fix missing dependencies in Ubuntu RPM builds?

If rpmbuild reports missing dependencies, install them using **`sudo apt install package-name`**. If converting RPM to DEB with Alien, ensure the required libraries are available. Use **`ldd binary-file`** to check for missing

shared libraries.

Is Ubuntu good for building RPM packages?

Ubuntu is not ideal for RPM packaging, but it works with additional setup. For serious RPM development, consider using a Red Hat-based system like Fedora or CentOS. However, if you only need occasional RPM builds, Ubuntu with rpmbuild and Alien can be sufficient.

Final Thoughts

This guide covered installing rpmbuild on Ubuntu, setting up a build environment, writing a spec file, building an RPM, and converting it to DEB for installation. If you need to distribute software across multiple Linux distributions, understanding both RPM and DEB packaging is essential.

ABOUT THE AUTHOR

As Editor in Chief of HeatWare.net, Sood draws on over 20 years in Software Engineering to offer helpful tutorials and tips for MySQL, PostgreSQL, PHP, and everyday OS issues. Backed by hands-on work and real code examples, Sood breaks down Windows, macOS, and Linux so both beginners and power-users can learn valuable insights. For questions or feedback, he can be reached at sood@heatware.net.

Google Tips iPhone Tips Laravel Tips Samsung Tips Software Testing (QA)

Find us on social

About Us

Welcome to HeatWare.net - A technology blog for geeks and non-geeks alike! We programming tips, code examples, Cloud technology help, Windows & application troubleshooting, and more!

Learn more about us.

Privacy Policy
Terms of Use